

Funciones en JavaScript: Los Bloques de Construcción Esenciales

Domina los fundamentos de la programación funcional en JavaScript y lleva tus habilidades al siguiente nivel

¿Qué es una Función en JavaScript?



Bloque Reutilizable

Las funciones son bloques de código que puedes usar múltiples veces en diferentes partes de tu programa



Organización Eficiente

Permiten estructurar y organizar el código de manera lógica, mejorando la legibilidad y mantenibilidad



Entradas y Salidas

Reciben datos (parámetros) como entrada, procesan la información y devuelven resultados específicos

Declaración de Funciones: La Forma Clásica

Las funciones tradicionales se definen usando la palabra clave `function`, seguida del nombre, paréntesis para parámetros y llaves para el cuerpo del código.

```
function saludar(nombre) {  
  return "Hola, " + nombre + "!";  
}
```

```
function sumar(a, b) {  
  return a + b;  
}
```

```
function calcularArea(base, altura) {  
  const area = base * altura / 2;  
  return area;  
}
```

Variables en Funciones: var, let y const

1

var: Scope Global y Función

Variable de función con hoisting. Evita usarla en nuevos proyectos por problemas de scope.

```
function ejemploVar() {  
  var mensaje = "Hola";  
  if (true) {  
    var mensaje = "Adiós";  
  }  
  console.log(mensaje); // Adiós  
}
```

2

let: Scope de Bloque

Variable de bloque moderna. Recomendada para variables que cambian durante la ejecución.

```
function ejemploLet() {  
  let contador = 0;  
  for (let i = 0; i < 3; i++) {  
    contador++;  
  }  
  console.log(contador); // 3  
}
```

3

const: Constantes Inmutables

Constantes de bloque. Usa para valores que no cambian y estructuras de datos estáticas.

```
function ejemploConst() {  
  const PI = 3.14159;  
  const usuario = {  
    nombre: "Juan",  
    edad: 25  
  };  
  // usuario = {}; Error!  
}
```

Funciones Flecha (Arrow Functions)

Sintaxis Compacta

Forma moderna y concisa introducida en ES6. Ideal para funciones de una sola línea.

```
const saludar = (nombre) => {  
  return "Hola, " + nombre;  
};
```

// Versión más corta

```
const saludar2 = nombre =>  
  "Hola, " + nombre;
```

No Tiene 'this' Propio

Hereda el valor de `this` del contexto circundante, útil en callbacks.

```
const usuario = {  
  nombre: "Ana",  
  saludar: () => {  
    return "Hola, " + this.nombre;  
  }  
};
```

Funciones con Return: Devolviendo Valores



Recibe Parámetros

Los datos de entrada que la función procesará



Procesa Lógica

Ejecuta el código y cálculos necesarios



Devuelve Resultado

Retorna un valor específico al llamador

```
function multiplicar(a, b) {  
  const resultado = a * b;  
  return resultado; // Devuelve el producto  
}
```

```
function obtenerEdad(anioNacimiento) {  
  const anioActual = new Date().getFullYear();  
  return anioActual - anioNacimiento;  
}
```

```
function esMayorDeEdad(edad) {  
  return edad >= 18; // Retorna true o false  
}
```

Funciones Comunes y Útiles



Funciones Matemáticas

Operaciones aritméticas, cálculos complejos y manipulación de números.

```
function calcularPotencia(base,
exponente) {
  return Math.pow(base,
exponente);
}

function redondear(numero,
decimales) {
  return Math.round(numero *
Math.pow(10, decimales)) /
Math.pow(10, decimales);
}
```



Manipulación de Texto

Transformación, validación y formateo de cadenas de texto.

```
function
capitalizarPrimeraLetra(texto) {
  return
texto.charAt(0).toUpperCase() +
texto.slice(1);
}

function validarEmail(email) {
  return email.includes("@") &&
email.includes(".");
}
```



Procesamiento de Arrays

Operaciones comunes para trabajar con listas y colecciones de datos.

```
function filtrarPares(numeros) {
  return numeros.filter(num => num
% 2 === 0);
}

function sumarArray(numeros) {
  return numeros.reduce((total,
num) => total + num, 0);
}
```

Mejores Prácticas en Funciones

01

Nombres Descriptivos

Usa verbos que indiquen claramente qué hace la función:

`calcularTotal()`, `validarUsuario()`

02

Single Responsibility

Cada función debe hacer una sola cosa y hacerla bien. Evita funciones que intenten hacer múltiples tareas

03

Parámetros con Valores por Defecto

Facilita el uso de funciones haciendo algunos parámetros opcionales con valores predeterminados

04

Documentación Clara

Agrega comentarios o JSDoc para explicar el propósito, parámetros, retorno y efectos secundarios

```
function calcularPrecioFinal(precio, descuento = 0, impuesto = 0.15) {  
  const precioConDescuento = precio - (precio * descuento);  
  return precioConDescuento * (1 + impuesto);  
}
```


Errores Comunes a Evitar

✗ Olvidar el Return

Las funciones sin return devuelven `undefined`. Asegúrate de retornar el valor esperado.

✗ Modificar Parámetros Globales

Evita modificar variables fuera del scope de la función. Usa parámetros e return para comunicación.

✗ Funciones Demasiado Largas

Funciones extensas son difíciles de entender y mantener. Divide en funciones más pequeñas y específicas.

Conclusión: Domina las Funciones

Reutilización

Las funciones te permiten escribir código una vez y usarlo múltiples veces, reduciendo duplicación

Organización

Divide tu código en bloques lógicos y manejables que son más fáciles de entender y mantener

Flexibilidad

Combina funciones tradicionales, flecha y técnicas modernas para escribir código eficiente y claro

Practica creando funciones de diferentes tipos, experimenta con `var`, `let` y `const`, y domina el uso del `return` para convertirte en un programador JavaScript experto.